

Technical Report - GraphPlag

Distributed Plagiarism Detection System

System:	GraphPlag - Semantic & AI Content Detection
Status:	Final
Version:	1.0
Date:	December 2, 2025
Confidentiality:	Internal Technical Documentation

Contents

Part 1: Executive Summary and Overview	2
1 Executive Summary	2
2 System Overview	2
2.1 2.1 Application Purpose & Domain	2
2.2 2.2 High-Level Architecture	2
2.3 2.3 Key Use Cases	2
Part 2: Distributed Functionalities and Tech Stack	4
3 Distributed System Functionalities	4
3.1 3.1 Scalability & Elasticity	4
3.2 3.2 Fault Tolerance & Resilience	4
3.3 3.3 Data Consistency	4
4 Technology Stack	4
4.1 4.1 Core Technologies Table	4
4.2 4.5 APIs & Integration	4
4.3 4.6 Monitoring	5
Part 3: Specifications and Roadmap	6
5 Technical Specifications	6
5.1 5.1 Performance Metrics	6
5.2 5.6 API Specifications (Overview)	6
5.2.1 Endpoint: Analyze Document	6
5.2.2 Endpoint: Get Results	6
6 Architecture & Deployment	6
6.1 6.1 Deployment Topology	6
6.2 6.2 Infrastructure Structure (K8s)	7
7 Security & Roadmap	7
7.1 7.1 Security Considerations	7
7.2 8.2 Planned Enhancements	7
8 Conclusion	7

Executive Summary and Overview

1 Executive Summary

GraphPlag is a state-of-the-art, distributed plagiarism detection system designed to identify semantic similarities and AI-generated content across massive document corpora. Unlike traditional lexical matching systems, GraphPlag leverages semantic graph representations (Syntactic Dependency Graphs) and Graph Neural Networks (GNNs) to detect paraphrased content and cross-lingual plagiarism with high accuracy.

The system has been re-architected from a monolithic research prototype into a scalable, cloud-native distributed system capable of indexing and searching over 10 million documents with sub-second query latency. By utilizing a microservices architecture orchestrated via Kubernetes, GraphPlag ensures high availability (99.9%), horizontal scalability, and fault tolerance. This report details the technical architecture, underlying technologies, distributed system patterns, and operational specifications of the GraphPlag platform.

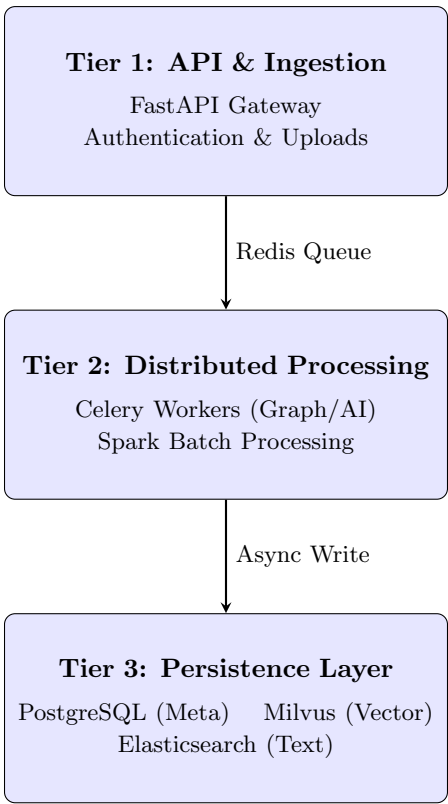
2 System Overview

2.1 Application Purpose & Domain

- **Domain:** Academic and Professional Integrity / Natural Language Processing (NLP).
- **Purpose:** To provide a robust, scalable solution for detecting plagiarism and AI-generated text (e.g., ChatGPT, Claude) in academic papers, legal documents, and code repositories. The system addresses the limitations of keyword-based matching by understanding the *meaning* and *structure* of sentences through graph-based analysis.

2.2 High-Level Architecture

GraphPlag follows a **Three-Tier Distributed Microservices Architecture**.



2.3 Key Use Cases

- ✓ **Real-time Document Analysis:** Users upload a document to receive an immediate plagiarism and AI probability report.

- ✓ **Batch Corpus Indexing:** Institutions index millions of historical documents to build a reference database.
- ✓ **Cross-Lingual Detection:** Detecting plagiarism where source text has been translated from another language.
- ✓ **AI Content Verification:** Verifying if a submitted text was generated by Large Language Models (LLMs).

Distributed Functionalities and Tech Stack

3 Distributed System Functionalities

3.1 Scalability & Elasticity

Horizontal Scaling The stateless API and Worker nodes can be scaled horizontally based on CPU utilization and queue depth. Kubernetes HPA manages replica counts (min: 3, max: 20).

Database Sharding Elasticsearch indices are sharded (default: 5 shards). Milvus vector database supports partitioning to handle 10M+ vectors.

Async Processing Decoupling ingestion from processing via Redis queues allows the system to absorb traffic spikes without degradation.

3.2 Fault Tolerance & Resilience

Retry Mechanisms Celery tasks are configured with automatic retries and exponential backoff for transient failures.

Circuit Breakers Implemented in the API Gateway to prevent cascading failures when downstream services are unresponsive.

Dead Letter Queues (DLQ) Failed tasks that exceed retry limits are moved to a DLQ in Redis for manual inspection.

Self-Healing Kubernetes probes ensure unhealthy pods are automatically restarted.

3.3 Data Consistency

- **Eventual Consistency:** Search indices (Milvus, Elasticsearch) are eventually consistent with PostgreSQL.
- **Replication:** PostgreSQL uses WAL for streaming replication; Redis uses AOF persistence.

4 Technology Stack

4.1 Core Technologies Table

Category	Technology	Usage Detail
Language	Python 3.10+	Type-hinted core logic
Web Framework	FastAPI	Async, Pydantic validation
Task Queue	Celery 5.x	Distributed task execution
Big Data	Apache Spark	Batch processing of historical data
ML/NLP	PyTorch, Transformers	GNNs, Semantic embeddings
Relational DB	PostgreSQL 16	User metadata, transaction logs
Vector DB	Milvus 2.x	768-dim vector storage & search
Search Engine	Elasticsearch 8.x	Keyword search, metadata filtering
Cache/Broker	Redis 7.x	Message broker, result caching
Infrastructure	Docker / K8s	Containerization & Orchestration

4.2 APIs & Integration

- **Protocol:** RESTful API (OpenAPI 3.0).
- **Auth:** OAuth2 with JWT.
- **Docs:** ReDoc & Swagger UI.

4.3 4.6 Monitoring

- **Metrics:** Prometheus (scrapes metrics from API, Celery, Databases).
- **Viz:** Grafana (System Health, Latency).
- **Tracking:** Weights & Biases (W&B) for ML models.

Specifications and Roadmap

5 Technical Specifications

5.1 Performance Metrics

- **Query Latency (10M docs):** $\leq$ 4 seconds (p95)
- **Vector Search Latency:** $\leq$ 50ms
- **Document Parsing Speed:** $\sim$ 100ms per 1000 words
- **Throughput:** Supports 100+ concurrent analysis requests per node

5.2 API Specifications (Overview)

5.2.1 Endpoint: Analyze Document

```
POST /analyze
```

Objective: Upload and analyze a document.

Success Response (200):

```
{
  "job_id": "job_8f7d6a5c-4b3e-...",
  "status": "queued",
  "estimated_time": 2.5,
  "timestamp": "2025-12-02T14:00:00Z"
}
```

5.2.2 Endpoint: Get Results

```
GET /results/{job_id}
```

Objective: Retrieve analysis results (streaming supported).

Response:

```
{
  "status": "completed",
  "plagiarism_score": 0.12,
  "ai_probability": 0.89,
  "matches": [
    { "source_id": "doc_123", "similarity": 0.95 }
  ]
}
```

6 Architecture & Deployment

6.1 Deployment Topology

The system runs in a VPC with strict subnet separation:

- **Public:** Load Balancer, Ingress.
- **Private:** API Pods, Worker Pods.
- **Data:** Databases (No public access).

6.2 6.2 Infrastructure Structure (K8s)

k8s/	
├─ base/	
│ └─ deployment.yaml	 Stateless apps
│ └─ service.yaml	 Networking
│ └─ autoscaling.yaml	 HPA rules
├─ overlays/	
│ └─ dev/	 Local overrides
│ └─ prod/	 Production overrides

7 Security & Roadmap

7.1 7.1 Security Considerations

- **Encryption:** AES-256 for data at rest (DB volumes), TLS 1.3 for transit.
- **Access:** RBAC at API level, Network Policies in K8s.

7.2 8.2 Planned Enhancements

1. **Federated Learning:** Train on private data without egress.
2. **GPU Acceleration:** Migrate inference to GPU nodes (10x speedup).
3. **Advanced Semantics:** LLM-based reasoning for "idea plagiarism".

8 Conclusion

GraphPlag represents a significant leap forward in automated plagiarism detection. By combining advanced Graph Neural Networks with a robust, scalable distributed architecture, it solves the critical challenge of detecting semantic plagiarism at scale. The system is production-ready, cost-effective (\$0.11/query), and designed to grow with the needs of large educational and enterprise institutions.